TenderoBot — Reporte técnico de avance 2

Oct 20, 2026 · commit `dc7a2e5`

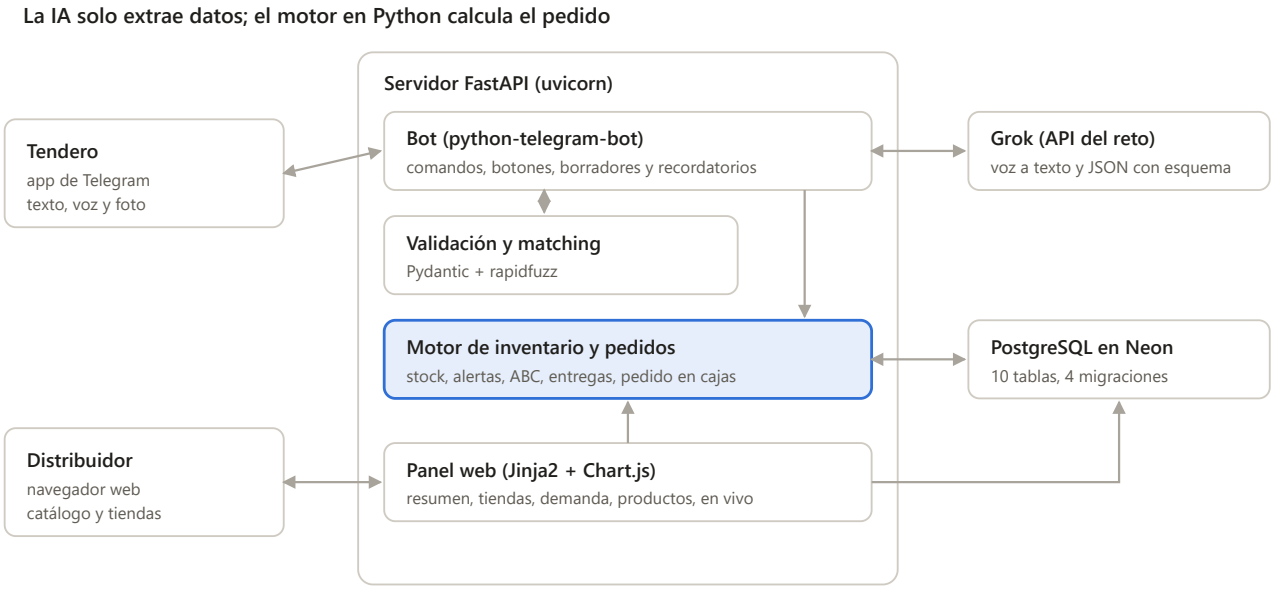
Resumen

El bot de Telegram @CO_Tendero_Bot y el panel web del distribuidor corren en local sobre un solo servidor FastAPI, conectados a Grok (a través de la API del reto) y a una base PostgreSQL en Neon. Las 119 pruebas automáticas pasan. Este avance cierra las fases 1 a 5 del plan de 6: además del panel, el bot ahora maneja días de entrega con hora límite, recordatorios automáticos, un pedido que el tendero edita por texto o voz y el catálogo del distribuidor con existencias. Falta el despliegue en Render (fase 6).

Fase	Alcance	Estado
1	Estructura, configuración, modelos, migraciones y seed	Terminada; 3 migraciones nuevas
2	Motor de inventario y pruebas	Terminada; ahora con entregas y hora límite
3	Cliente Grok, matching y esquemas	Terminada; nueva extracción de cambios al pedido
4	Bot de Telegram con todos los flujos	Terminada; pedido editable, días de entrega, recordatorios y `/catalogo`
5	Panel web del distribuidor	Terminada; resumen, tiendas, demanda, productos y actualización en vivo
6	Docker, despliegue y README	README listo; Docker y despliegue pendientes

Arquitectura y stack

Un solo proceso FastAPI aloja el bot, el motor de inventario, el panel web y la tarea de recordatorios. Grok y Neon son los únicos servicios externos.



arquitectura · un servidor, dos servicios externos

El bot guarda ventas, conteos y pedidos en Neon solo después de que el tendero confirma con un botón. El motor lee de Neon para `/stock`, `/alertas`, `/pedido` y para el panel. Stack: Python 3.12, FastAPI, python-telegram-bot 22.8, SQLAlchemy 2.1 asíncrono con asyncpg, Alembic, Pydantic 2, httpx, rapidfuzz, Jinja2, Chart.js, bcrypt y pytest.

Modelo de datos

Siguen siendo diez tablas en PostgreSQL. Este avance agrega tres migraciones que solo suman columnas; ninguna borra datos. Toda consulta sigue filtrando por `store_id` o `distributor_id`.

Migración	Tabla	Columnas nuevas
`0002_delivery_days`	stores	`route_weekdays` (días en que el proveedor pasa por la zona), `delivery_days_set_at`
`0003_order_delivery`	stores, suggested_orders	`last_reminder_for` (evita repetir el recordatorio), `delivery_date` del pedido
`0004_product_availability`	products	`available_units` (vacío = sin límite), con restricción `CHECK >= 0`

El seed (`scripts/seed.py`) ahora carga solo datos base: 1 distribuidor, el usuario `demo@demo.co`, 3 tiendas sin vincular (MARTA01, PARQUE02, LUZ03) y 30 productos con alias coloquiales. Ya no genera ventas sintéticas: todo lo que muestra el panel llega desde Telegram. Volver a correrlo borra los datos del distribuidor.

Motor de inventario

Las reglas de stock, venta diaria, desviación, cobertura, stock de seguridad, ABC, riesgo y estancado no cambian. Cambia cómo se calcula el horizonte, que ahora depende de las entregas reales de cada tienda (`delivery_schedule()` en `app/inventory/engine.py`).

Concepto	Regla
Días de entrega	El tendero los escoge en el bot entre los días de ruta del proveedor. Sin configurar se asume el lunes.
Hora límite	6 p. m. hora Colombia, `lead_time_days` días antes de la entrega (mínimo 1).
Próxima entrega	La primera entrega cuya hora límite todavía no ha pasado.
Horizonte	Días desde hoy hasta la entrega siguiente a la próxima: lo que se pide hoy debe durar hasta que llegue el pedido siguiente.
Ajuste a existencias	El pedido nunca supera las cajas completas que el distribuidor tiene (`cap_to_available()`). Sin cantidad registrada no hay límite.
Cambios del tendero	Funciones puras en `app/inventory/orders.py`: agregar, quitar o fijar cantidad; las unidades sueltas se redondean hacia arriba a cajas.

Extracción con Grok y matching

Grok (modelo `grok-4.20-0309-non-reasoning`, `temperature=0`) solo convierte voz, texto o foto en JSON; nunca decide cantidades a pedir. El cliente (`app/ai/grok_client.py`) usa `httpx` asíncrono contra la API del reto, compatible con OpenAI.

1. Voz: el endpoint `/stt` la convierte a texto en español y luego se extrae como un mensaje de texto.
2. Texto y foto: `/chat/completions` con `response_format: json_schema`. La foto viaja en base64 dentro del mismo mensaje, junto al catálogo de la tienda (SKU, nombres y alias).
3. Cada respuesta se valida con Pydantic. Ante error o JSON inválido hay 1 reintento; si vuelve a fallar se lanza `ExtractError` y el tendero ve "No te entendí, intenta de nuevo o escríbelo".
4. `matching.py` descarta SKU que no estén en el catálogo y busca por nombre y alias con rapidfuzz (`WRatio`, umbral 80). Sin coincidencia, la línea no se guarda.
5. El tendero confirma con un botón; solo entonces se escribe en la base.

Extracción	Salida	Llamadas y tokens de entrada (aprox.)
Ventas (texto)	`{sku, product_text, quantity}`	1 llamada · ~800
Ventas (voz)	Igual que texto	2 llamadas (`/stt` + extracción) · ~800 + audio

Extracción	Salida	Llamadas y tokens de entrada (aprox.)
Conteo (foto)	`{sku, quantity, confidence}`	1 llamada · ~800 + imagen
Cambios al pedido	`{sku, product_text, action, quantity, unit}`	1 llamada · ~1.000

Las instrucciones aceptan ventas escritas sin la palabra "vendí" y prohíben interpretar una venta como un cambio al pedido. `/stock`, `/alertas`, `/pedido`, `/recibido`, `/catalogo`, los recordatorios y el panel no llaman a Grok.

Bot de Telegram

El bot (python-telegram-bot 22.8, asíncrono) corre dentro de FastAPI: polling en local y webhook en producción. Los handlers siguen organizados por grupos: rate limit, vinculación, comandos y botones, texto de ventas y respuesta por defecto.

Entrada	Respuesta del bot
`/start` + código de tienda	Vincula el chat, muestra el aviso de la Ley 1581 de 2012 y, al aceptar, pide escoger los días de entrega entre los días de ruta.
Texto o nota de voz ("2 coca litro y un pan")	Resumen de ventas con Confirmar / Cancelar; guarda solo al confirmar.
Foto del estante	Conteo estimado, corregible por texto ("coca litro 8").
`/stock`, `/alertas`	Stock actual; productos en riesgo y estancados con su motivo.
`/pedido`	Pedido para la próxima entrega, en cajas, con motivo, total en COP y hora límite. Queda como borrador por 6 horas.
"quita el arroz", "ponle 2 cajas más de coca"	Ajusta el borrador y lo vuelve a mostrar. Lo que no está en el catálogo se informa y no se agrega.
Botón "Enviar al distribuidor"	Bloquea los productos con `SELECT … FOR UPDATE`, ajusta a existencias, descuenta unidades y guarda el pedido. Si pasó la hora límite, lo envía para la entrega siguiente y avisa.
`/recibido`	Registra la llegada del último pedido enviado y suma la mercancía.
`/catalogo`	Productos del proveedor con precio por unidad y cuáles están agotados.
Automático: día de la hora límite	Recordatorio entre las 9 a. m. y la hora límite, una sola vez por entrega, si la tienda no ha pedido. La tarea revisa cada 10 minutos.
Automático: cambio en el catálogo	Aviso de precio nuevo o producto nuevo; varias novedades llegan en un solo mensaje.

Panel web del distribuidor

Login con bcrypt y cookie de sesión firmada. Cada ruta valida que la tienda o el producto sean del distribuidor; si no, responde 404 sin revelar que existen.

Ruta	Qué muestra o hace
`/panel`	Resumen: tiendas en riesgo, productos más críticos y pedidos por despachar.
`/panel/tiendas` y `/panel/tiendas/{id}`	Estado de cada tienda; detalle con ventas de 28 días, pedido sugerido ajustado a existencias, pedidos enviados y clasificación ABC.
`/panel/demanda`	Demanda agregada de la próxima semana, en cajas.
`/panel/productos` (`/nuevo` , `/guardar`)	Crear productos y editar precio, existencias y activo. Todo el catálogo se guarda de una vez o no se guarda nada; los cambios se avisan a los tenderos en segundo plano.
`/panel/productos/{id}/ventas`	Ventas por día de la semana de las últimas 8 semanas, de todas las tiendas o de una (JSON para el gráfico).
`/panel/actualizacion`	Huella de los datos que llegan de Telegram. El navegador la consulta cada 8 segundos y recarga solo si cambió.

Pruebas y verificación

Las 119 pruebas de pytest pasan (`python -m pytest -q`). Las de IA usan respuestas simuladas, sin llamar a la API.

Archivo	Pruebas
`tests/test_engine.py` : stock, cobertura, ABC, entregas y hora límite, redondeo a cajas	26 aprobadas
`tests/test_catalog.py` : validación del formulario y cambios del catálogo	22 aprobadas
`tests/test_orders.py` : cambios al pedido, ajuste a existencias y recordatorios	21 aprobadas
`tests/test_bot_gate.py` : vinculación, consentimiento y días de entrega	18 aprobadas
`tests/test_ai.py` : validación, reintento y matching con Grok simulado	16 aprobadas
`tests/test_web.py` : formato, agregaciones, login y rutas protegidas	16 aprobadas

No está cubierto por pruebas automáticas: el recorrido completo en Telegram (voz, foto, botones y recordatorios). Se prueba a mano.

Configuración y seguridad

Los secretos viven solo en `.env`, excluido de git. Variables: `TELEGRAM_BOT_TOKEN`, `TELEGRAM_WEBHOOK_SECRET`, `PUBLIC_BASE_URL`, `DATABASE_URL`, `GROK_API_KEY`, `GROK_BASE_URL`, `GROK_MODEL`, `BOT_MODE`, `SESSION_SECRET` y `DEMO_PASSWORD`.

- Límite de 20 mensajes por minuto por chat en el bot.
- Login del panel: máximo 5 intentos por minuto por IP, con cuenta regresiva en pantalla.
- Fotos de máximo 10 MB y audios de máximo 2 minutos; se procesan en memoria y no se guardan.
- Los logs no incluyen chats, mensajes ni datos del tendero; el de `httpx` está en WARNING porque sus URLs llevan el token.
- Webhook protegido con secreto, comparado con `hmac.compare_digest`.
- Bloqueo de filas al enviar pedidos, para que dos tiendas no se lleven la misma última caja.

Pendientes

Para la demo falta el despliegue; hoy el bot y el panel solo responden mientras el servidor corre en el PC local.

- [ ] Dockerfile para Render, despliegue en modo webhook y UptimeRobot contra `/health` cada 5 minutos.
- [ ] Aviso al distribuidor cuando un tendero envía un pedido; hoy solo queda en el panel.
- [ ] Registrar los tokens que reporta Grok en cada llamada para medir el costo real por tarea.
- [ ] Datos de prueba con tendencia (el producto que crece).
- [ ] Herramientas, ejecuciones y casos de evaluación de las siguientes semanas del reto.